Comparing Inverse Kinematics for VR-Controlled 6-DOF Robot

Filippo Marcantoni
Worcester Polytechnic Institute
Worcester, MA, USA
fmarcantoni@wpi.edu

Manav Mepani
Worcester Polytechnic Institute
Worcester, MA, USA
mmepani@wpi.edu

Pranay Katyal
Worcester Polytechnic Institute
Worcester, MA, USA
pkatyal@wpi.edu

Prahladh Raja
Worcester Polytechnic Institute
Worcester, MA, USA
pnamboorkrishnam@wpi.edu

Anirudh Ramanathan
Worcester Polytechnic Institute
Worcester, MA, USA
aramanathan@wpi.edu

Prasham Soni
Worcester Polytechnic Institute
Worcester, MA, USA
psoni@wpi.edu

***Abstract*—The goal of this project is to implement the fundamentals of an autonomous method that provides a dynamic and flexible point of view for a humanoid being tele-manipulated performing a grasping task, addressing challenges such as occlusion in complex environments [1], [2]. This method allows the user to directly change the position of a non-grasping arm's end-effector using a Virtual Reality (VR) headset, while the orientation of the camera mounted to it is autonomously adjusted to be pointing at the end-effector of the arm performing the grasping task [3]. The minimal deliverable of this project is a comparison study to compare the performances of the different algorithms discussed in the related work section [4], [5].**

***Index Terms*—Inverse Kinematics Algorithms, Particle Swarm Optimization (PSO), Singular Value Decomposition Damped Least Squares (SVD-DLS), Improved DLS, Newton Raphson, 6-DOF robot, VR teleoperation, real-time control, algorithm benchmarking**

I. Introduction

A. Project Overview

This project investigates real-time inverse kinematics (IK) algorithms for a 6-DOF robotic manipulator controlled through a virtual reality (VR) headset. The goal is to evaluate the performance of four IK methods in determining the joint configurations necessary to move the end-effector to a desired target pose. The target position is dynamically inferred from the operator's head movements: looking left/right (yaw) controls the x-axis, while looking up/down (pitch) controls the y-axis displacement of the end-effector.

We developed a layered architecture across two machines. A Windows machine running Unity captures the headset pose and publishes it to a ROS topic, while an Ubuntu machine receives this data, extracts the pitch and yaw angles, and maps them to displacement values in MATLAB, which simulates the manipulator (see Section III-A).

The interaction begins from a home configuration. As the user looks in a specific direction, counters track how far and for how long the head is moved away from the center. Once the user returns to the home orientation, the counters are converted into x and y displacements. This target pose is passed to all four IK algorithms simultaneously to enable fair benchmarking under identical conditions, rather than using a continuous stream of points. Algorithms are evaluated based on three metrics: computation time, cumulative joint-space distance, and weighted joint movement (see Section IV).

Our methodology selection process considered several IK approaches from literature:

- **Improved Damped Least Squares (I-DLS)**: Enhances convergence smoothness and computational efficiency for visual grasping [5].
- **SVD-Damped Least Squares (SVD-DLS)**: Uses singular value decomposition to improve numerical stability near singularities [2].
- **Offset Modification (OM)**: Iterative geometric simplification for closed-form solutions [1].
- **Manipulability-Aware IK**: Resolved motion rate control using manipulability measures [3].
- **Soft Computing Techniques**: Metaheuristics like PSO, Artificial Bee Colony, and Firefly Algorithm for high-DOF systems [6].

While promising, some methods were excluded due to practical constraints. Manipulability-Aware IK required complex singularity avoidance strategies incompatible with our VR mapping system. Soft computing metaheuristics (except PSO) exhibited excessive computational latency for real-time use. OM's geometric simplification proved difficult to integrate with our displacement-based target generation. We ultimately selected four algorithms (Newton-Raphson, I-DLS, SVD-DLS, and PSO) that balanced computational efficiency, singularity robustness, and compatibility with our VR-driven control paradigm (see Section II).

II. Algorithms Used

The following section introduces the four algorithms compared in our study, highlighting their strengths and relevance to inverse kinematics in robotics. Each method is detailed below with a focus on its mathematical formulation and implementation relevance.

A. Algorithm 1: Newton-Raphson (NR)

The Newton-Raphson method is a classical approach used for solving nonlinear systems. In the context of IK, it operates by updating joint angles using the pseudoinverse of the Jacobian based on the difference between the current end-effector position and the target:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + J^{-1}(\mathbf{q}_k)(\mathbf{x}_{\text{target}} - \mathbf{x}_k) \tag{1}$$

where $J(\mathbf{q}_k)$ is the Jacobian matrix at the current joint configuration and $\mathbf{x}_k$ is the current end-effector position. This method requires computation of the inverse of the Jacobian and assumes the function is well-behaved near the current estimate. While fast under well-conditioned scenarios, it can struggle near singularities and requires a good initial guess [2], [1].

B. Algorithm 2: Improved Damped Least Squares (I-DLS)

In the original Damped Least Squares (DLS) algorithm, there are limitations such as failure to reach target positions outside of the workspace and no joint safety checks. The Improved Damped Least Squares algorithm makes two changes to the original DLS method. These are the joint limits and the clamping techniques.

The joint limits technique helps to limit the angle of a joint and prevents any mechanical damage to the actuator used to control the joint and gives a more natural motion for it. The technique restricts the joint angles to be within a minimum and maximum value. This avoids jerky and unsafe postures of the robot arm, such as singular configurations [5].

The clamping technique helps reach unreachable target positions. This is done by replacing the unreachable target position with a nearly reachable target position within the workspace of the robot. This helps for better convergence values, even with noisy data and reduces computation time [5].

The positional error is first clamped to avoid unstable steps:

$$\mathbf{e} = \mathbf{x}_{\text{target}} - \mathbf{x}_{\text{current}}, \quad \mathbf{e} = \begin{cases} \mathbf{e} \cdot \frac{D_{\max}}{\|\mathbf{e}\|}, & \text{if } \|\mathbf{e}\| > D_{\max} \\ \mathbf{e}, & \text{otherwise} \end{cases} \tag{2}$$

Dynamic damping is computed as:

$$\lambda = \begin{cases} \lambda_0(1 - \frac{\omega}{\omega_0}), & \text{if } \omega < \omega_0 \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

The joint update is then given by:

$$\Delta\mathbf{q} = J^\top(JJ^\top + \lambda^2 I)^{-1}\mathbf{e} \tag{4}$$

Finally, a joint-limit-aware scaling factor α is applied to ensure safety:

$$\mathbf{q}_{\text{new}} = \mathbf{q}_{\text{current}} + \alpha\Delta\mathbf{q} \tag{5}$$

Where:

- $D_{\max}$ is the maximum error magnitude for scaling.
- λ is the dynamic damping factor based on ω.
- λ_0 is the initial damping factor when $\omega < \omega_0$.
- ω_0 is the threshold for the manipulability measure ω.
- $\omega = \sqrt{\det(JJ^\top)}$ is the manipulability measure.
- J is the Jacobian matrix.
- $\Delta\mathbf{q}$ is the joint configuration change.
- I is the identity matrix.

C. Algorithm 3: The Singular Value Decomposition-Damped Least Squares (SVD-DLS)

The paper titled "The Singular Value Decomposition: Computation and Applications to Robotics" proposes the use of Singular Value Decomposition (SVD) as a robust framework for addressing inverse kinematics and analyzing robot manipulators [2]. By decomposing the Jacobian matrix into orthogonal components, SVD enables the computation of a stable pseudoinverse, even in configurations close to singularities. The paper also employs the Direct Linear Transformation (DLT) method to solve homogeneous systems of equations, which frequently arise in calibration and constraint-solving tasks. This is achieved by extracting the singular vector corresponding to the smallest singular value. Furthermore, the authors introduce an enhanced approach called SVD-Damped Least Squares (SVD-DLS), which integrates damping directly into the singular values. This modification improves convergence speed and stability, especially near singularities, making it highly suitable for real-time robotic applications.

The Jacobian J is defined as:

$$J = \frac{\partial\mathbf{x}}{\partial\mathbf{q}} \in \mathbb{R}^{3\times n} \tag{6}$$

where:

- J is the analytical Jacobian matrix of the manipulator,
- $\mathbf{q} \in \mathbb{R}^n$ is the joint configuration vector,
- $\mathbf{x} \in \mathbb{R}^3$ is the end-effector position.

Using singular value decomposition:

$$J = U\Sigma V^\top \tag{7}$$

where:

- $U \in \mathbb{R}^{3\times 3}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal matrices,
- $\Sigma \in \mathbb{R}^{3\times n}$ is a diagonal matrix with singular values σ_i.

The damped pseudoinverse is constructed as:

$$\Sigma_i^\dagger = \frac{\sigma_i}{\sigma_i^2 + \lambda^2}, \quad J^\dagger = V\Sigma^\dagger U^\top \tag{8}$$

where:

- $\lambda = \lambda_0 + \omega_0\|\mathbf{e}\|$ is a dynamically adjusted damping factor,
- $\mathbf{e} = \mathbf{x}_{\text{target}} - \mathbf{x}_{\text{current}}$ is the error in end-effector position.

The joint update rule becomes:

$$\Delta\mathbf{q} = J^\dagger\mathbf{e} \tag{9}$$

This is applied iteratively until the error norm $\|\mathbf{e}\|$ is within a set threshold (see Section II-C for details).

D. Algorithm 4: Particle Swarm Optimization (PSO)

Soft computing techniques, especially nature-inspired metaheuristic algorithms, offer powerful alternatives to traditional analytical methods for solving inverse kinematics (IK) problems in robotics [6], [4]. These techniques are more tolerant to uncertainties and nonlinearities, making them suitable for

real-time applications and high degree-of-freedom (DOF) manipulators.

Among these, Particle Swarm Optimization (PSO) has shown significant promise due to its simplicity, flexibility, and strong convergence capabilities in high-dimensional search spaces. PSO is inspired by the social behavior of birds and fish, where each agent (particle) adjusts its position in the search space by learning from both its own experience and that of neighboring agents [6].

The IK problem involves finding the joint configuration $\mathbf{q} \in \mathbb{R}^n$ such that the end-effector reaches a desired position $\mathbf{p}_{\text{target}}$. In PSO, each particle represents a candidate joint configuration. The algorithm evolves a swarm of such particles over time using the following velocity and position update equations:

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1 r_1 \left(\mathbf{p}_i^{\text{best}} - \mathbf{q}_i^{(t)}\right) + c_2 r_2 \left(\mathbf{g}^{\text{best}} - \mathbf{q}_i^{(t)}\right) \tag{10}$$

$$\mathbf{q}_i^{(t+1)} = \mathbf{q}_i^{(t)} + \Delta t \cdot \mathbf{v}_i^{(t+1)} \tag{11}$$

Where:

- $\mathbf{q}_i$ is the position (joint angles) of the i-th particle.
- $\mathbf{v}_i$ is the velocity of the i-th particle.
- $\mathbf{p}_i^{\text{best}}$ is the best position found so far by the i-th particle.
- $\mathbf{g}^{\text{best}}$ is the global best position found by the entire swarm.
- w is the inertia weight (linearly decayed from 0.9 to 0.4).
- c_1, c_2 are cognitive and social learning coefficients (scheduled during iterations).
- r_1, r_2 are random values between 0 and 1.
- Δt is the time step.

The objective function minimized is the Euclidean distance between the current end-effector position (obtained via forward kinematics) and the target point:

$$\text{Error}_i = \left\|\mathbf{p}_i^{\text{current}} - \mathbf{p}_{\text{target}}\right\|_2 \tag{12}$$

Particle Swarm Optimization (PSO) offers several advantages for motion planning in robotic systems. First, its global search capability—unlike gradient-based methods—does not depend on initial guesses, which makes PSO especially robust in non-convex or poorly conditioned search spaces [4]. Second, the algorithm's convergence behavior can be tailored through time-step and inertia scheduling, allowing an initial phase of broad exploration followed by gradual exploitation of promising regions. Finally, PSO naturally accommodates constraint handling: by incorporating custom techniques for enforcing joint limits directly into the update rules, it ensures that all candidate solutions remain feasible throughout the optimization process.

To ensure feasibility, joint limits are enforced using a reflection mechanism, where particles bouncing off joint boundaries invert their velocities and are repositioned within limits [6].

Each of these inverse kinematics algorithms offers unique advantages and trade-offs in terms of speed, robustness, and adaptability to constraints such as joint limits or singularities. While analytical and Jacobian-based methods like Newton-Raphson (Section II-A) and SVD-DLS (Section II-C) are favored for their speed and precision in well-behaved scenarios, they can struggle in the presence of complex constraints or unreachable targets. Conversely, optimization-based approaches like PSO (Section II-D) excel in handling nonlinearity and high-dimensionality but often at the cost of increased computation time. By comparing these diverse methods within a consistent framework and using a VR-based control interface (see Section III), our study aims to provide insights into their real-world applicability for intuitive and responsive robot teleoperation.

III. Methodology

After individually implementing and validating the four IK algorithms in MATLAB [5], [2], [6], we designed a benchmarking framework to compare them using identical target poses (see Algorithms II-A–II-D). The benchmark function accepts a desired end-effector pose and evaluates each algorithm's performance based on: computation time, cumulative distance traveled in joint space, and weighted joint-angle changes.

A. Headset Mapping Architecture

The mapping between headset movements and the displacement of the end-effector in the x and y directions was achieved through a layered architecture. This consisted of a TCP link between a Windows machine (running Unity on a Meta Quest 2 to publish '/headsetInfo' via the ROS_TCP_Connector) and an Ubuntu host (running the 'unity-repository' and 'oculus _ros' packages to subscribe to '/headsetInfo' and republish '/yaw_motor_pid/setpoint' and '/pitch_motor_pid/setpoint'). A MATLAB node (via the ROS Toolbox) on Ubuntu then subscribes to those topics to feed the IK routines.

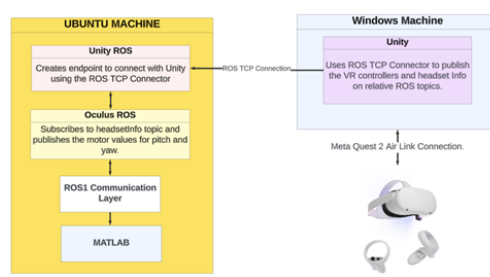

Fig. 1. System architecture for VR-to-robot mapping.

The received pitch and yaw values drive incremental counters based on user gaze (e.g., looking up increments pitch). Once the user returns to the home orientation, the accumulated counters are scaled and added to the x–y target pose for the non-grasping arm. To prevent interference during computation, ROS subscribers are briefly shut down upon return to home pose, the benchmark is executed, and results (as defined in Section III) are displayed.

IV. Evaluation

This evaluation protocol follows the headset mapping described in Section III-A and the benchmarking framework of Section III. The system was tested on four movements—upward, rightward, downward, and leftward—where the user maintains gaze for approximately 10s in each direction before returning to the home pose to trigger the IK computation. All four IK algorithms are executed per trial:

- Newton–Raphson (Section II-A, [1]),
- Improved Damped Least Squares (Section II-B, [5]),
- SVD-Damped Least Squares (Section II-C, [2]),
- Particle Swarm Optimization (Section II-D, [6]).

The benchmarking function measures computation time, cumulative distance traveled in joint space, and weighted joint-angle changes for each algorithm. The aggregated performance metrics across all four gaze-directed movements are shown in Figure 2.

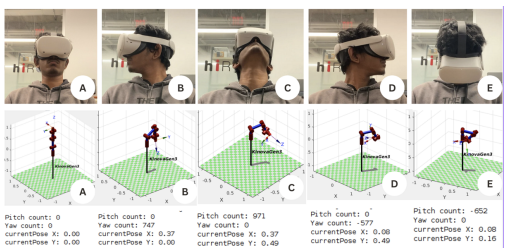

Fig. 2. Performance metrics for each IK algorithm across the four gaze-directed movements.

V. Discussion & Conclusion

The experiment setup above confirmed our initial belief. Analytical and Jacobian-based approaches, such as Newton-Raphson (Section II-A) and SVD-DLS (Section II-C), are often preferred for their computational efficiency and accuracy in structured and well-conditioned environments [1], [2]. However, their performance can degrade in the presence of complex joint constraints or when attempting to reach infeasible poses. On the other hand, optimization-based methods like Particle Swarm Optimization (PSO, Section II-D) are more robust to nonlinearities and high-dimensional search spaces, though this typically comes at the expense of higher computational cost [6], [4].

By evaluating these varied techniques within a unified framework and interfacing them with a VR-based control system (Sections III–IV), this study assesses their practical effectiveness in enabling smooth and intuitive robot teleoperation. Figures 3, 4, and 5 summarize our findings.

The performance comparison of these methods confirms the distinct trade-offs between deterministic and stochastic approaches. As shown in Figure 3, PSO exhibits robust coverage of diverse target configurations due to its population-based search strategy. However, this generality comes with significant drawbacks: PSO incurs the highest completion time and cumulative joint distance (Figure 4), suggesting inefficiencies

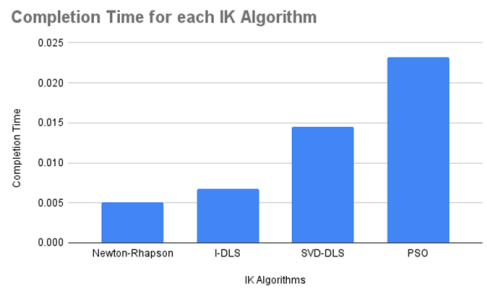

Fig. 3. Computation time for each IK algorithm.

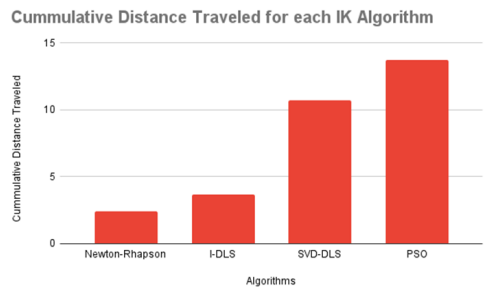

Fig. 4. Cumulative distance traveled by the end-effector for each IK algorithm.

for real-time use. Additionally, PSO often produces greater joint movements (Figure 5), which can lead to increased actuator wear and energy consumption [6].

Deterministic approaches like NR, I-DLS, and SVD-DLS exhibit much lower computation times and more conservative joint movements (Table I). Newton-Raphson and SVD-DLS maintain a good balance between speed and joint efficiency but may struggle under poor initial guesses [1]. Among these, I-DLS (Section II-B) offers a favorable compromise by maintaining low joint displacement while avoiding excessive computation [5].

An interesting observation was that when the user's head movement was restricted to the X-axis (yaw-only), the analytical solvers (Sections II-A, II-C) failed to return valid joint updates, whereas PSO still produced feasible solutions. This indicates that analytical methods require fully defined end-

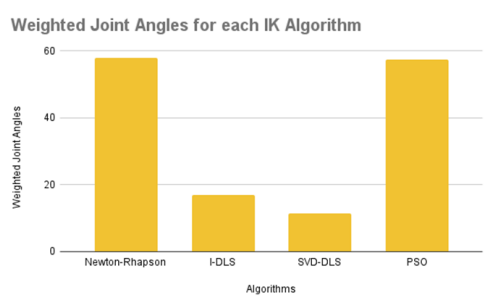

Fig. 5. Weighted joint-angle changes for each IK algorithm.

TABLE I
COMPARISON OF INVERSE KINEMATICS (IK) ALGORITHMS

Metric	NR	I-DLS	SVD-DLS	PSO
Convergence Speed	High	High	Moderate	Low
Robustness to Initial Guess	Low	Moderate	Moderate	Very High
Computation Time	Low	Low	Moderate	High
Residual Error Tolerance	Moderate	High	High	High
Ease of Implementation	Moderate	Moderate	Complex	High
Best Use Case	Well-conditioned tasks	Constrained environments	Near-singular configs	Global search, bad initial guesses

effector poses (both x and y displacements) for convergence, while PSO can approximate joint configurations from partial target information [6].

Overall, while PSO demonstrates robustness in globally searching complex or constrained inverse kinematics (IK) spaces, deterministic methods like SVD-Damped Least Squares (SVD-DLS) exhibit superior performance in real-time teleoperation scenarios, particularly near nominal configurations. The SVD-DLS method (Section II-C) stands out due to its ability to stabilize convergence near singularities through singular value damping, a feature critical for maintaining smooth motion in local workspaces. By dynamically adjusting the damping factor based on the manipulability measure ω, SVD-DLS achieves a balance between computational efficiency and singularity avoidance, making it uniquely suited for applications requiring precise, real-time adjustments—such as the camera alignment task in this study.

VI. FUTURE WORK

In this project, our initial objective was to align the end-effector's orientation such that it consistently points toward the grasping arm. However, due to time constraints, we were unable to implement this feature. In future work, this can be achieved by incorporating a logic that iteratively samples points around the target position of the non-grasping arm and evaluates which configuration results in the Z-axis of the camera frame (assumed to be aligned with the camera's pointing direction) being most closely aligned with the origin of the grasping arm's end-effector frame.

This could leverage the Improved Damped Least Squares (I-DLS) method (Section II-B) for joint-limit-aware optimization or extend the SVD-Damped Least Squares (SVD-DLS) approach (Section II-C) to handle orientation constraints near singular configurations [2]. Additionally, hybrid strategies combining Particle Swarm Optimization (PSO, Section II-D) for global exploration with deterministic solvers for local refinement could further enhance real-time performance [6], [4].

REFERENCES

[1] G. Z. Grudic and P. D. Lawrence, "Iterative inverse kinematics with manipulator configuration control," *IEEE Trans. Robot. Autom.*, vol. 9, no. 4, pp. 476–483, 1993.

[2] A. A. Maciejewski and C. A. Klein, "The singular value decomposition: Computation and applications to robotics," *Int. J. Robot. Res.*, vol. 8, no. 6, pp. 63–79, 1989.

[3] G. Marani, J. Kim, J. Yuh, and W. K. Chung, "A real-time approach for singularity avoidance in resolved motion rate control of robotic manipulators," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 2, 2002, pp. 1973–1978.

[4] S. Luo, D. Chu, Q. Li, and Y. He, "Inverse kinematics solution of 6-dof manipulator based on multi-objective full-parameter optimization PSO algorithm," *Front. Neurorobot.*, vol. 16, p. 791796, 2022.

[5] M. R. Hadi Setyawan, R. S. Dewanto, B. S. Marta, E. H. Binugroho, and D. Pramadihanto, "Kinematics modeling of six degrees of freedom humanoid robot arm using improved damped least squares for visual grasping," *Int. J. Electr. Comput. Eng.*, vol. 13, no. 1, 2023.

[6] G. B. Mahanta, B. B. V. L. Deepak, M. Dileep, B. B. Biswal, and S. K. Pattanayak, "Prediction of inverse kinematics for a 6-dof industrial robot arm using soft computing techniques," in *Soft Computing for Problem Solving*. Springer Singapore, 2019, pp. 519–530.

[7] F. Zonfrilli, G. Oriolo, and D. Nardi, "A biped locomotion strategy for the quadruped robot Sony ERS-210," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 3, 2002, pp. 2768–2774.